

Computational graphs & the maths of AI

Lecture 3 – Three tools: linear algebra, calculus, and computation graphs

Dr Thomas Robinson

August 5, 2026

| Part 1 · Linear algebra

Why we need this

A modern neural network operates on **tensors with millions of entries**. We need notation that scales — both for writing and for computing.

Every layer of a neural network is essentially a *matmul* followed by a non-linear activation function applied element-wise.

But we're social scientists 🤔

- Matrices aren't that scary! Stacking survey into a data frame (n rows, k columns) gives you a matrix.
- We're largely dressing up familiar maths using new notation.

Scalars

The smallest unit. Just numbers.

$$1, 1.5, -0.001, \pi$$

Combine them with arithmetic to make expressions:

$$y = ((2 \times 3) + 5) \times 7 = 77$$

Or write the same thing as a **function**:

$$y = f(x) = (2x + 5) \times 7$$

A function is just a name for "input x , compute these steps, output y ."

Functions can be **composed**, i.e. $f(x) = g(h(x))$ where $h(x) = 2x + 5$ and $g(z) = 7z$.

Composition is the whole idea behind a deep network.

Vectors

A 1-D ordered list of scalars. Two we'll keep using:

$$\mathbf{a} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}$$

Addition / subtraction: element-wise, requires matching length.

$$\mathbf{a} + \mathbf{b} = \begin{bmatrix} 1 + 4 \\ 2 + 5 \\ 3 + 6 \end{bmatrix} = \begin{bmatrix} 5 \\ 7 \\ 9 \end{bmatrix}$$

Scalar multiplication: multiply every entry.

$$3\mathbf{a} = \begin{bmatrix} 3 \\ 6 \\ 9 \end{bmatrix}$$

A survey-stats example

Rescaling everyone's height from cm to m is scalar multiplication of the height column

Row vs column — and the transpose

A vector can be a *column* $\begin{bmatrix} a_1 \\ a_2 \end{bmatrix}$ or a *row* $\begin{bmatrix} a_1 & a_2 \end{bmatrix}$.

The **transpose** ($'$ or T) swaps them:

$$\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} \iff \mathbf{a}^T = \begin{bmatrix} a_1 & a_2 \end{bmatrix}$$



Note

The transpose is **only a re-orientation**. The numbers don't change. It exists so the operations later in this lecture line up dimension-wise.

Matrices

A 2-D array of scalars. Two indices — row and column.

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Addition is again element-wise:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} - \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} -4 & -4 \\ -4 & -4 \end{bmatrix}$$

A weight matrix in a neural network is just a 2-D table of numbers, ordered by “which output node” and “which input feature”.

Indexing a matrix

ROW FIRST, THEN COLUMN — ALWAYS

We write a_{ij} for the entry in **row** i , **column** j :

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

SINGLE ENTRIES

- $a_{12} = 2$
- $a_{31} = 7$
- $a_{23} = 6$

A whole row

Fix $i = 2$:

$$\begin{bmatrix} 4 & 5 & 6 \end{bmatrix}$$

A whole column

Fix $j = 2$:

$$\begin{bmatrix} 2 \\ 5 \\ 8 \end{bmatrix}$$



$a_{12} = 2$ but $a_{21} = 4$ — **different entries**. Row always comes first, and this is the most common slip in the rest of this lecture.

TRY

What is a_{32} ? And what shape is column 3?

ANSWER

$a_{32} = 8$ — row 3, column 2. Column 3 is $\begin{bmatrix} 3 \\ 6 \\ 9 \end{bmatrix}$, a 3×1 column vector.

Indexing in maths vs. indexing in code

Maths starts at 1

a_{11} is the **top-left** entry.

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

So $a_{23} = 6$.

Python starts at 0

```
A[0, 0]    # 1    ← top-left
A[1, 2]    # 6    ← same entry as a_23
A[1]       # row 2: [4, 5, 6]
A[:, 1]    # column 2: [2, 5, 8]
```



Important

The **order is the same** — row, then column. Only the *starting number* differs: a_{ij} is `A[i-1, j-1]`.

You met this in yesterday's lab; you'll use it every day from here on.

Transposing a matrix

Swap rows and columns:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}, \quad \mathbf{A}^T = \begin{bmatrix} a_{11} & a_{21} \\ a_{12} & a_{22} \\ a_{13} & a_{23} \end{bmatrix}$$

TRY

What is $(\mathbf{A}^T)^T$?

ANSWER

A. Two transposes get you back where you started.

The dot product

THE MOST IMPORTANT OPERATION WE'LL SEE TODAY

For two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$ (same length!):

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i$$

A **single number**. Sometimes written $\langle \mathbf{a}, \mathbf{b} \rangle$ or $\mathbf{a}^T \mathbf{b}$.

Worked example. Let $\mathbf{a} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}$.

$$\mathbf{a} \cdot \mathbf{b} = (1)(4) + (2)(5) + (3)(6) = 4 + 10 + 18 = 32$$

Why it matters.

A neuron's pre-activation is $\sum_i w_i x_i + b$. That sum is a **dot product** $\mathbf{w} \cdot \mathbf{x}$.

- A weighted sum of survey responses is a dot product.
- A weighted average is a normalised dot product.

Matrix multiplication

Take $\mathbf{A} \in \mathbb{R}^{n \times k}$ and $\mathbf{B} \in \mathbb{R}^{k \times t}$.

The product $\mathbf{AB}$ is the $n \times t$ matrix whose entry (i, j) is the **dot product of row i of $\mathbf{A}$ and column j of $\mathbf{B}$** :

$$(\mathbf{AB})_{ij} = \sum_{\ell=1}^k A_{i\ell} B_{\ell j}$$

Dimension rule (memorise this): the *inner* dimensions must match.

$$\underbrace{(n \times k)}_{\mathbf{A}} \cdot \underbrace{(k \times t)}_{\mathbf{B}} \longrightarrow (n \times t)$$

If they don't match, the product is *undefined*.

Matrix multiplication — worked example

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$\mathbf{AB} = \begin{bmatrix} \underbrace{1 \cdot 5 + 2 \cdot 7}_{=19} & \underbrace{1 \cdot 6 + 2 \cdot 8}_{=22} \\ \underbrace{3 \cdot 5 + 4 \cdot 7}_{=43} & \underbrace{3 \cdot 6 + 4 \cdot 8}_{=50} \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Two warnings.

- In general $\mathbf{AB} \neq \mathbf{BA}$. Order matters.
- Dimensions must match, i.e. if $\mathbf{A} \in \mathbb{R}^{3 \times 2}$, $\mathbf{B} \in \mathbb{R}^{2 \times 4} \rightarrow \mathbf{AB}$ is 3×4 . But $\mathbf{BA}$ is undefined.



$\mathbf{AB} \neq \mathbf{BA}$

Compute $\mathbf{BA}$ for the above matrices by hand. You should get a different number than $\mathbf{AB}$. This mismatch is why we need transposes to keep things orderly.

Why a layer is a matrix product

Yesterday: a hidden layer of three nodes, each computing $\sum_i w_i^{(h)} x_i + b^{(h)}$ for the same input $\mathbf{x}$.

Stack the per-node weights into a matrix $\mathbf{W} \in \mathbb{R}^{h \times k}$ and biases into a vector $\mathbf{b} \in \mathbb{R}^{1 \times h}$. The entire layer's pre-activation is:

$$\mathbf{x}\mathbf{W}^T + \mathbf{b} \in \mathbb{R}^{1 \times h}.$$

The transpose makes the inner dimensions match: $(1 \times k) \cdot (k \times h) \rightarrow (1 \times h)$.

I.e. One output per h nodes

Why a transpose? — dimensional intuition

Suppose $h = 2$ (two hidden nodes) and $k = 3$ (three input features).

- Weights for node 1: $\begin{bmatrix} w_1^{(1)} & w_2^{(1)} & w_3^{(1)} \end{bmatrix}$
- Weights for node 2: $\begin{bmatrix} w_1^{(2)} & w_2^{(2)} & w_3^{(2)} \end{bmatrix}$
- Stack as $\mathbf{W} = \begin{bmatrix} w_1^{(1)} & w_2^{(1)} & w_3^{(1)} \\ w_1^{(2)} & w_2^{(2)} & w_3^{(2)} \end{bmatrix}$ — shape (2×3) .

To compute $\mathbf{x} \cdot \mathbf{W}^T$ with $\mathbf{x}$ a row vector of length 3, we need $\mathbf{W}^T$ of shape (3×2) .

Put it together:

$$\mathbf{W}^T = \begin{bmatrix} w_1^{(1)} & w_1^{(2)} \\ w_2^{(1)} & w_2^{(2)} \\ w_3^{(1)} & w_3^{(2)} \end{bmatrix}$$

$$\text{For a given } \mathbf{x} = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix}, \quad \mathbf{x} \cdot \mathbf{W}^T = \begin{bmatrix} (x_1 w_1^{(1)} + x_2 w_2^{(1)} + x_3 w_3^{(1)}) & (x_1 w_1^{(2)} + x_2 w_2^{(2)} + x_3 w_3^{(2)}) \end{bmatrix}$$

So we get a two-length row vector output (one value per neuron).

Stop and check

STOP & CHECK — PAIR WITH A NEIGHBOUR, 2 MINUTES

QUESTION: WHAT HAPPENS IF $\mathbf{X}$ IS A DATAFRAME (I.E. MULTIPLE OBSERVATIONS)?

$\mathbf{W} \in \mathbb{R}^{50 \times 5}$ — a hidden layer of width 50 receiving 5 input features. $\mathbf{X} \in \mathbb{R}^{100 \times 5}$ — a dataset of 100 observations.

What is the shape of $\mathbf{XW}^T$? What does each row of it represent?

ANSWER

$(100 \times 5) \cdot (5 \times 50) \rightarrow (100 \times 50).$

Each row = the 50 hidden-layer pre-activations for one observation.

Notice this is the forward pass logic of yesterday

- But now, done simultaneously over every observation in one matmul!

| Plenary · matrix products by hand

Plenary · matrix products by hand

≈ 20 MIN · PEN AND PAPER, IN PAIRS

FIVE PROBLEMS

For each, compute or say “undefined” (with a reason).

1. $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 \\ 6 \end{bmatrix} = ?$

2. $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 7 & 8 \\ 9 & 10 \end{bmatrix} = ?$

3. $\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} = ?$

4. $\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \begin{bmatrix} 4 & 5 \end{bmatrix} = ?$

5. $\begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix} = ?$

Ten minutes. Then we'll discuss together.

Plenary · answers

1. $\begin{bmatrix} 17 \\ 39 \end{bmatrix}$. A $(2 \times 2) \cdot (2 \times 1) \rightarrow (2 \times 1)$ matrix-vector product. Think of it as two dot products stacked.
2. $\begin{bmatrix} 7 & 8 \\ 9 & 10 \end{bmatrix}$. The first matrix is the **identity**. Multiplying by it leaves the second matrix unchanged.
3. 32. A row $\times$ column dot product. Single number.
4. $\begin{bmatrix} 4 & 5 \\ 8 & 10 \\ 12 & 15 \end{bmatrix}$. A *column $\times$ row* — the outer product. Gives a matrix, not a scalar.
5. **Undefined**. Inner dimensions $2 \neq 3$ — can't do it.



Tip

The most common error in deep learning code is a shape mismatch. PyTorch raises a clear error when this happens. Train your eye now — and you'll save hours later.

| Part 2 · Differentiation

What we want — and why

Tomorrow, for any parameter w in our model, we will want to know:

If I *nudge* w slightly, does the loss go up or down — and by how much?

This information is a **derivative**.



You'll have seen derivatives before.

We're going to re-introduce them with two restrictions:

- We don't care about proofs!
- We do care about computation — i.e. *getting a number out* given a function and a value of x .

The derivative — slope of a function

For $y = f(x)$, the derivative $\frac{dy}{dx}$ is the **slope** of f at point x .

For $y = ax^b + c$ (where a, b, c are scalars):

$$\frac{dy}{dx} = b \cdot a x^{b-1}$$

- Constants drop out (their derivative is zero).
- $b = 1$: $y = ax + c \Rightarrow \frac{dy}{dx} = a$.
- $b = 2$: $y = ax^2 + c \Rightarrow \frac{dy}{dx} = 2ax$.

Worked example.

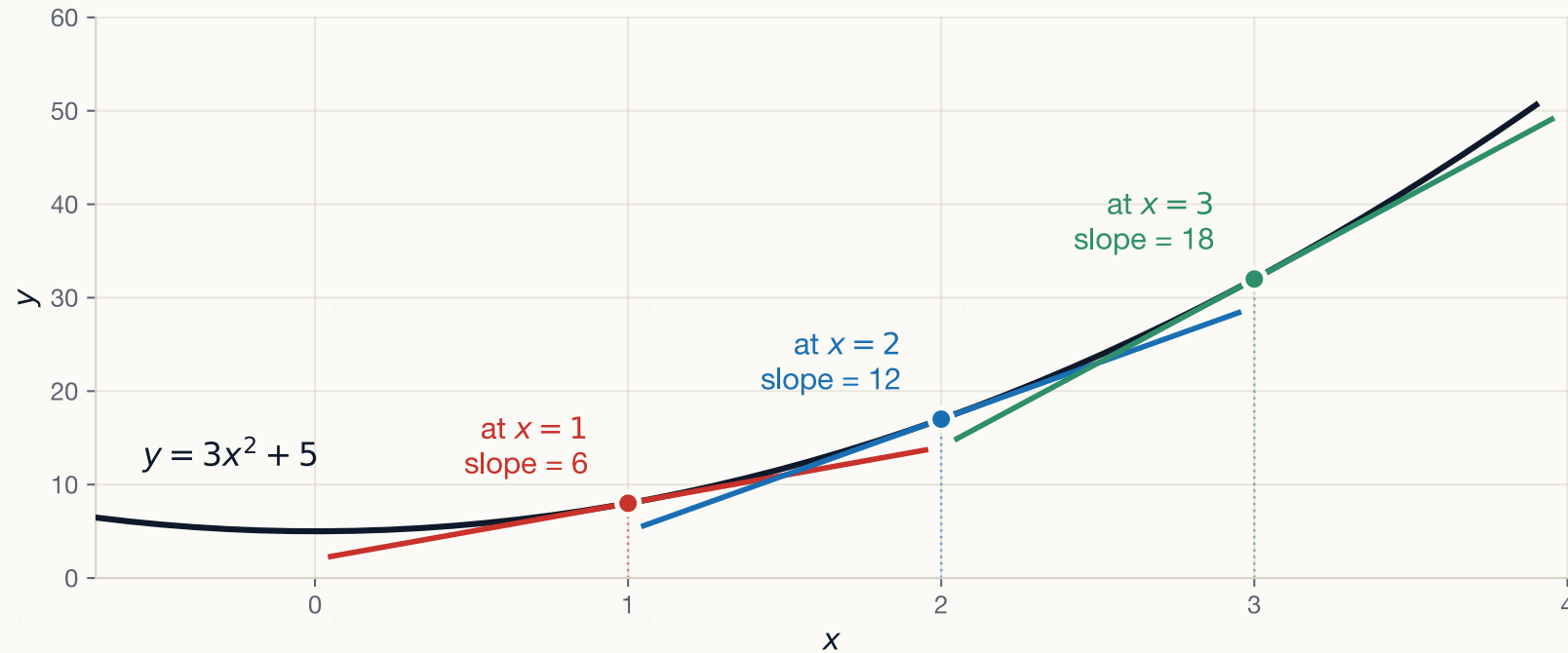
$$y = 3x^2 + 5.$$

$$\frac{dy}{dx} = 6x.$$

At $x = 1$: slope = 6. At $x = 2$: slope = 12. At $x = 0$: slope = 0 (local minimum!).

Seeing the slope

THE SAME WORKED EXAMPLE, DRAWN: $y = 3x^2 + 5$



The tangent is the straight line that just *touches* the curve at that point. Its steepness **is** the derivative there — so $\frac{dy}{dx} = 6x$ is a formula for the slope at **every** x , not one number. Steeper curve $\rightarrow$ bigger derivative.

A small cheat-sheet

Power rule

$$\frac{d}{dx} x^n = nx^{n-1}$$

Sum rule

$$\frac{d}{dx} (a + b) = \frac{da}{dx} + \frac{db}{dx}$$

Constant rule

$$\frac{d}{dx} c = 0$$

Exponential

$$\frac{d}{dx} e^x = e^x$$

$$\frac{d}{dx} e^{f(x)} = f'(x)e^{f(x)}$$

Natural log

$$\frac{d}{dx} \ln x = \frac{1}{x}$$

Sigmoid (useful later)

$$\frac{d}{dx} \sigma(x) = \sigma(x)(1 - \sigma(x))$$



Note

You will not be asked to derive these. You **will** be asked to use them — both in this course and in the lab tomorrow.

A harder function

What about $y = (2x + 3)^2$?

You *could* expand to $y = 4x^2 + 12x + 9$ and apply the power rule: $\frac{dy}{dx} = 8x + 12$.

But what if expanding is messy — or impossible? We need a **mechanical** rule that doesn't depend on cleverness.

That rule is **the chain rule**.

The chain rule

For $y = f(g(x))$ — a function inside a function — define $u = g(x)$ so $y = f(u)$. Then:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

Worked example — $y = (2x + 3)^2$.

- Let $u = 2x + 3$. So $y = u^2$.
- $\frac{dy}{du} = 2u$ (power rule on u^2).
- $\frac{du}{dx} = 2$ (power rule on $2x + 3$).
- $\frac{dy}{dx} = 2u \cdot 2 = 4(2x + 3) = 8x + 12$. 

Same answer, mechanically.

The intuition. A small change in x propagates first through the inner function (u changes a little) and then through the outer function (y changes *because* u changed).

We want to know what the change through g *and* f is so we **multiply the two rates together**.

The chain rule, generalised

For $y = f_n(f_{n-1}(\cdots f_1(x) \cdots))$ — a chain of n functions:

$$\frac{dy}{dx} = \frac{dy}{du_{n-1}} \cdot \frac{du_{n-1}}{du_{n-2}} \cdots \frac{du_1}{dx}$$

Each piece is a small, easy derivative. Multiply them. Done.

This is the deepest reason deep learning works: any reasonable composition of operations can be differentiated **one operation at a time**, then assembled.

A longer worked chain

TAKE A FUNCTION WITH THREE NESTED OPERATIONS:

$$y = \left(\frac{1}{n}(a - x)^2\right)^{1/2}$$

Decompose: $* u_2 = a - x * u_1 = u_2^2 = (a - x)^2 * y = (u_1/n)^{1/2}$

Apply the chain rule:

$$\frac{dy}{dx} = \underbrace{\frac{d}{du_1} \left[\left(\frac{u_1}{n} \right)^{1/2} \right]}_{\text{outer}} \cdot \underbrace{\frac{d}{du_2} [u_2^2]}_{\text{middle}} \cdot \underbrace{\frac{d}{dx} [a - x]}_{\text{inner}}$$

Three small derivatives. We don't even need to expand y to do this.

Partial derivatives & gradients

For a function of several variables, e.g. $y = f(x_1, x_2) = 3x_1^2 + 2x_2 + 1$:

Compute **partial derivatives** — treat the other vars as constants:

$$\frac{\partial y}{\partial x_1} = 6x_1, \quad \frac{\partial y}{\partial x_2} = 2$$

Collect them into a **gradient vector**:

$$\nabla f = \begin{bmatrix} \frac{\partial y}{\partial x_1} \\ \frac{\partial y}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 6x_1 \\ 2 \end{bmatrix}$$

Why we care.

For a neural network with millions of weights, the loss is a function $L(\mathbf{W})$ from “all the weights” to “a single number”.

The gradient ∇L is a vector of the same size as $\mathbf{W}$ — one entry per weight. It tells us, for *each* weight, how the loss changes.

That is the thing we minimise tomorrow.

| **Activity · chain-rule exercises**

Activity · chain rule

≈ 15 MIN · PEN AND PAPER, IN PAIRS

For each y , find $\frac{dy}{dx}$ — using the chain rule, *without* expanding.

1. $y = (3x + 1)^4$

2. $y = e^{2x+5}$

3. $y = \ln(x^2 + 1)$

4. $y = \sigma(3x)$ where $\sigma(z) = \frac{1}{1+e^{-z}}$

5. $y = (a - x)^2$ — find $\frac{dy}{dx}$, treating a as a constant.

Twelve minutes. We'll discuss the trickier ones.

Activity · answers

$$1. \frac{dy}{dx} = 4(3x + 1)^3 \cdot 3 = 12(3x + 1)^3.$$

$$2. \frac{dy}{dx} = e^{2x+5} \cdot 2 = 2e^{2x+5}.$$

$$3. \frac{dy}{dx} = \frac{1}{x^2 + 1} \cdot 2x = \frac{2x}{x^2 + 1}.$$

$$4. \frac{dy}{dx} = \sigma(3x)(1 - \sigma(3x)) \cdot 3.$$

$$5. \frac{dy}{dx} = 2(a - x) \cdot (-1) = -2(a - x).$$



Tip

Tomorrow we use exactly this pattern — repeatedly — to compute gradients across an entire neural network.

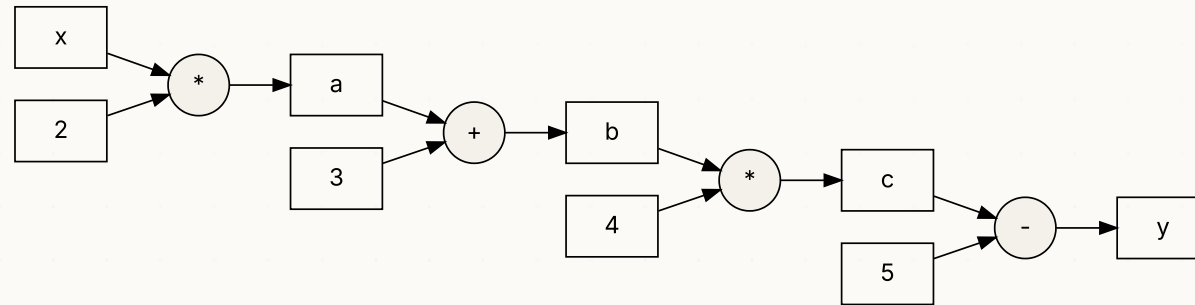
| Part 3 · Computational graphs

A function as a picture

Take $f(x) = 4(2x + 3) - 5$. It has:

- **Values:** $x, 2, 3, 4, 5$.
- **Operations:** $\times, +, -$.
- **Order** (BIDMAS).

Draw it as a graph — left-to-right, with operations as circles and values as boxes:



A **computational graph** is just a flow chart for a calculation. Each node either holds a value or performs an operation. Edges show the order of operations.

The pieces — value nodes and operation nodes

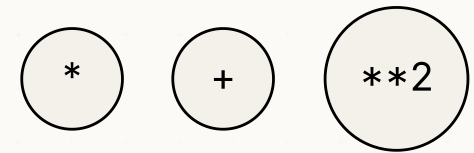
Value nodes

- Contain **data** — scalars, vectors, matrices, or higher-dimensional tensors.
- Will later also hold **gradients** (their slot for $\frac{\partial L}{\partial}$).
- Drawn as boxes.

x	data = 0.0	grad = 0.0
---	------------	------------

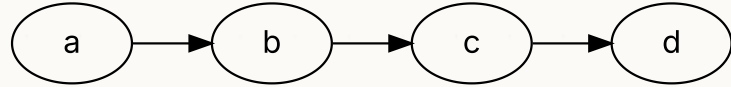
OPERATION NODES

- Apply a **function** to incoming values.
- Arrows in = arguments. Arrow out = result.
- Drawn as circles.

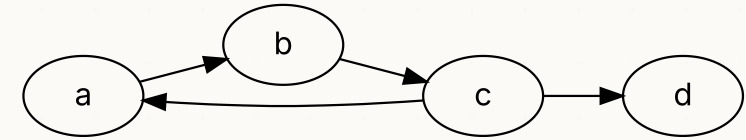


Acyclic — almost always

We use **directed acyclic graphs** (DAGs) — no operation's output ever flows back to an *earlier* node.



Acyclic — what we use



Cyclic — pathological for plain neural nets



Note

We **will** see cycles later — recurrent networks on Day 9, where the same operation is reused across time steps. But each unrolled time-step is still a DAG.

You've maybe seen this picture before

SOCIAL-SCIENCE GRAPHS

A computational graph is structurally identical to:

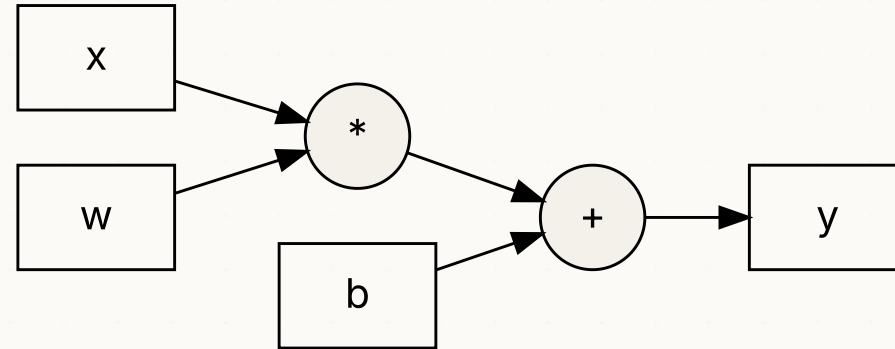
- A **causal DAG** in causal inference (cf. Judea Pearl's work).
- A **factor graph** in probabilistic modelling.
- A **dataflow diagram** in software engineering.
- A **path diagram** in structural equation models.

The shared idea: a calculation is a *structure of operations on values*.

The structure itself carries information, by telling us what depends on what, what we can compute first, and what to differentiate with respect to what.

The perceptron as a graph

Yesterday's linear perceptron $y = wx + b$ becomes:



(With an activation function σ added between the **+** and the output, you'd insert one more node — same picture, one more step.)

Every node has a local derivative we can compute — using the cheat-sheet from earlier. The chain rule will glue them.

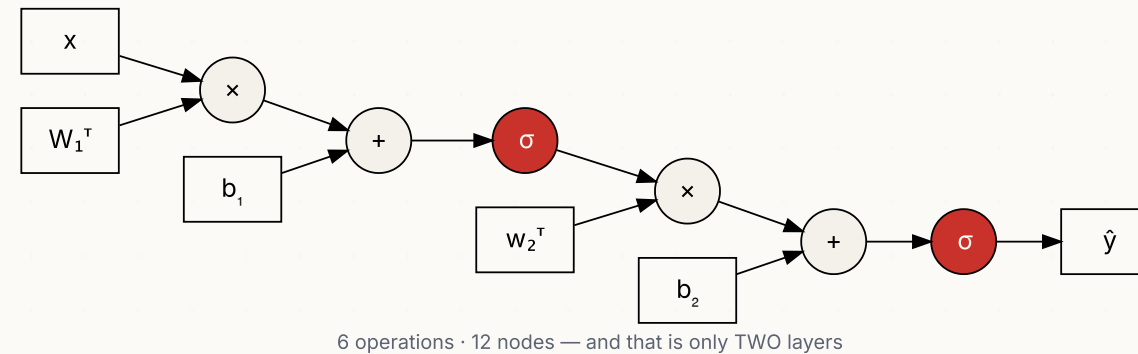
That's the move that makes the whole edifice work. Tomorrow we use it.

A network as a graph

Yesterday's two-layer network was:

$$\hat{y} = \sigma \left(\sigma \left(\mathbf{x} \mathbf{W}^{(1)\top} + \mathbf{b}^{(1)} \right) \mathbf{w}^{(2)\top} + b^{(2)} \right)$$

BUILD IT ONE OPERATION AT A TIME — WATCH THE NODE COUNT



We **multiply, add, “squash”**, twice. What grows is the *tracking*: three operations per layer, and every one needs its own local derivative.

In **Labs 3–4** you build exactly this, where every single arithmetic operation becomes a node in your **Value** graph. A tiny network makes a surprisingly big graph — and that is precisely why we let the computer walk it for us.

The two passes — preview of tomorrow

Forward pass

- Walk the graph **left-to-right**.
- At each node, compute its output using the operation and incoming values.
- Store the output (we'll need it).
- End: a prediction.

BACKWARD PASS

- Walk the graph **right-to-left**.
- At each node, compute $\frac{\partial L}{\partial(\text{this node})}$ using the chain rule and local derivatives.
- End: a gradient for every parameter.



Tip

This is what frameworks (PyTorch, JAX, TensorFlow) automate. Tomorrow you'll write a tiny version of it yourself, in Python.

Take stock

WHERE YOU ARE AT THE END OF DAY 3

You can now:

1. Multiply matrices and check shapes by hand.
2. Differentiate simple expressions, including with the chain rule.
3. Draw any expression — from a simple polynomial to a two-layer network — as a directed acyclic graph.
4. State what we still don't have: a **way to put loss into the graph and let gradients flow backwards**.



Tip

You have all three tools. Tomorrow we put them together.

| Lab brief · today's afternoon

Today's lab — 90 minutes

GOALS

In Python (no PyTorch yet), build a small **computational graph library** from scratch.

You will implement:

1. A `Value` class that stores `data`, `grad`, and a list of “child” values.
2. Methods `__add__`, `__mul__`, and `__pow__` that build a graph as you do arithmetic.
3. A small test: write `c = a * b + 1` for `Value`s `a`, `b`, and verify the graph has the right structure.

Deliverable. A working `Value` class. We will add gradient computation to it **tomorrow** — that step turns this library into a fully-working autodiff system.

The pattern is Andrej Karpathy's `micrograd`. We are walking it deliberately.

See you tomorrow

LECTURE 4 — BACKPROPAGATION

If you want a head start: skim Karpathy's *micrograd* GitHub README tonight. Don't read the implementation yet — we'll write it together tomorrow.